

Chap.7 순환 신경망 (Recurrent Neural Network)

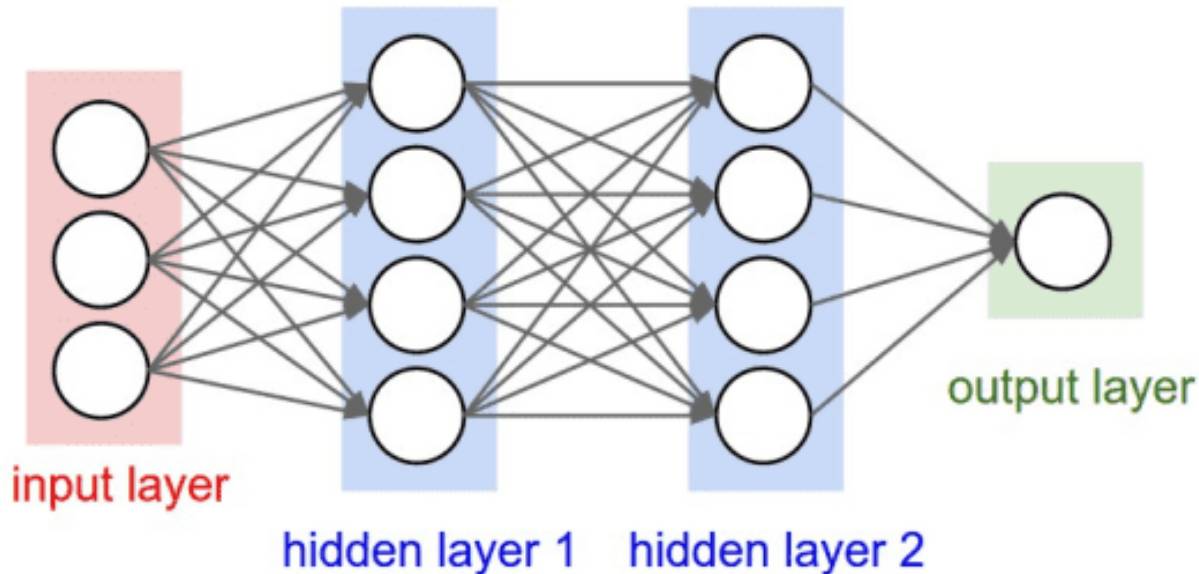
방 수 식 교수
(bang@tukorea.ac.kr)

한국공학대학교 전자공학부

2024년도 2학기
고급머신러닝실습 & 인공지능설계실습2

Feed Forward Neural Network

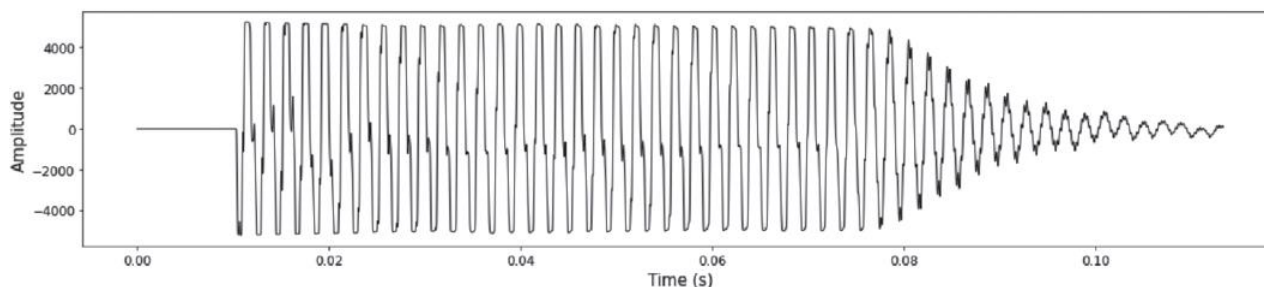
- 순 방향으로만 연결되어있는 Neural Network
 - Node 간의 연결이 순환(Recurrent)하거나 역순으로 가지 않음.



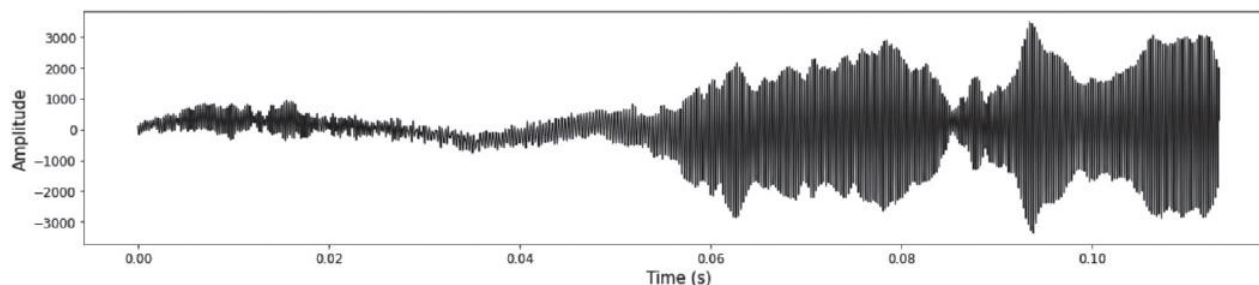
시계열 데이터 = time series data

■ 시계열 데이터란?

- 시간의 흐름에 따라 변화하는 데이터
- 일정 시간 간격으로, 지속적으로 측정되는 데이터
ex) 자이로 센서, 압력 센서, 마이크 등
- 일반적인 수치 데이터에 비해 데이터의 크기와 복잡도가 매우 높음



비프음



새소리

Recurrent Neural Network



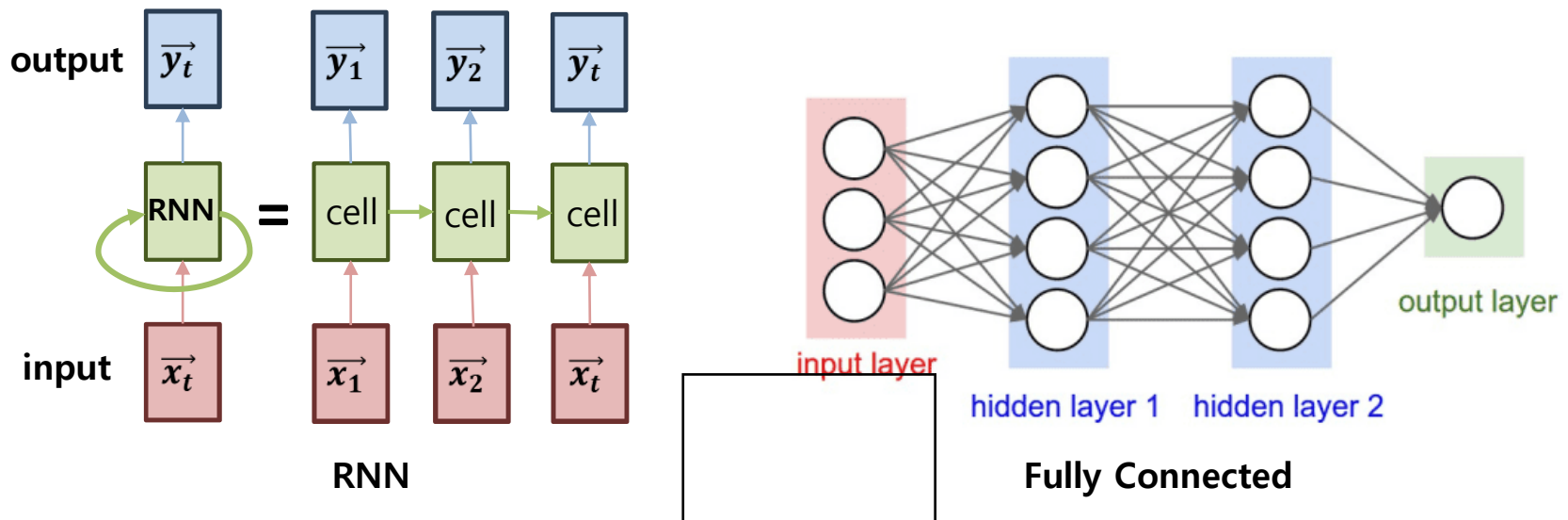
▪ RNN (순환 신경망)

- 길이가 계속 변하는 데이터의 처리를 위해 고안

ex) "순환 신경망이란 무엇인가"

$t = 1$ 길이: 2 $t = 2$ 길이: 5 $t = 3$ 길이: 4 t : 시간

- 완전 연결 신경망에 비해 적은 수의 Weight로 효과적으로 학습
- 연속되는 시계열 데이터를 다루기에 적합한 인공신경망
- 응용 분야 → 음성 인식, 감정 분석, 자동 번역 등등



RNN의 종류

■ One-to-Many

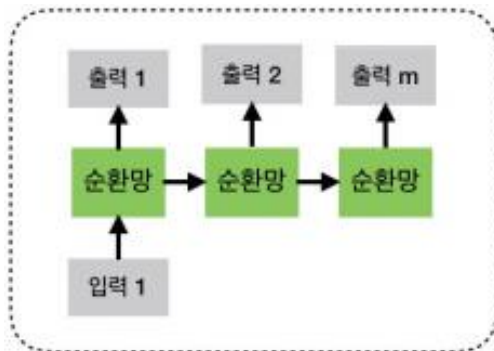
- ex) 하나의 이미지 입력에 대한 설명을 생성하는 신경망

■ Many-to-One

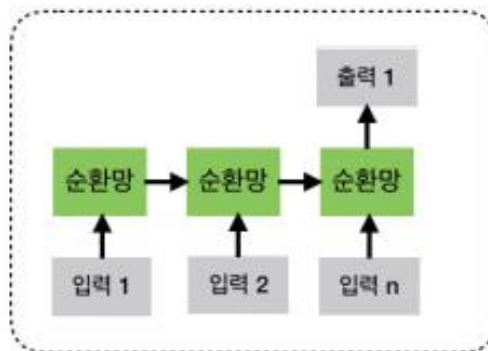
- ex) 입력 문장에 대해 문장의 감정을 분석하는 신경망

■ Many-to-Many

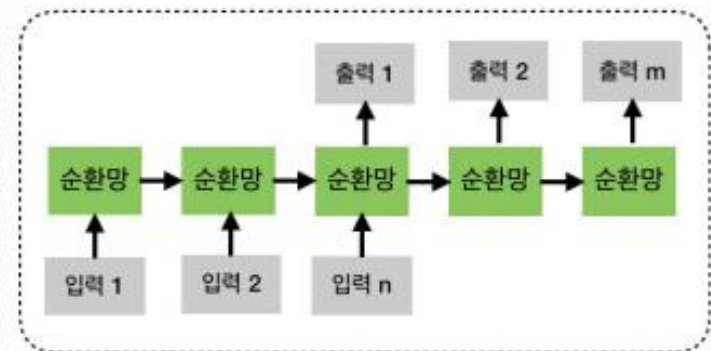
- ex) 입력 문장에 대한 대답을 하는 신경망 → 챗봇, 번역기



One-to-Many



Many-to-One



Many-to-Many
or Sequence to Sequence

RNN의 구조

Cell (Memory Cell, RNN Cell)

- RNN의 은닉층
- 활성화 함수를 통해 결과를 출력하고, 과거의 값을 기억하는 역할

Hidden State : h_t

- 현 시점의 Cell이 갖고 있는 정보
- 다음 시점으로 넘겨줄 정보

가중치 W 를 갖는 함수

시간 t 에서의 입력 벡터

bias

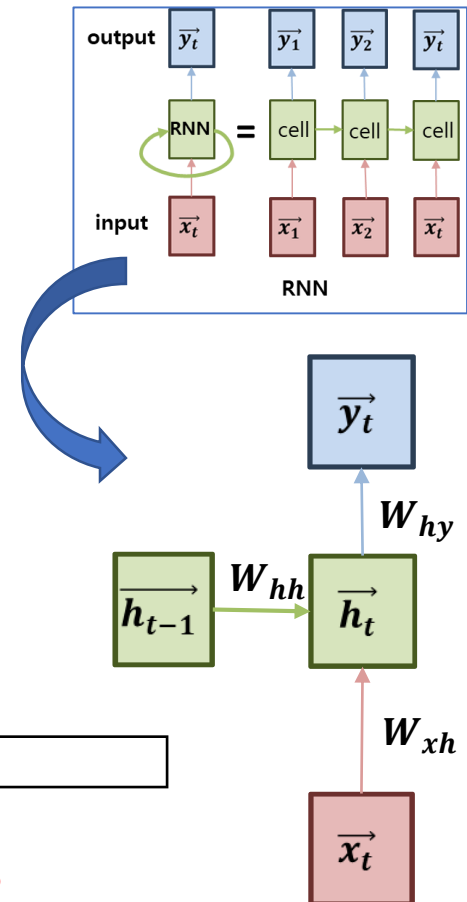
$$h_t = f_W(h_{t-1}, x_t) = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_n)$$

state ()

New state Old state

+ + bias

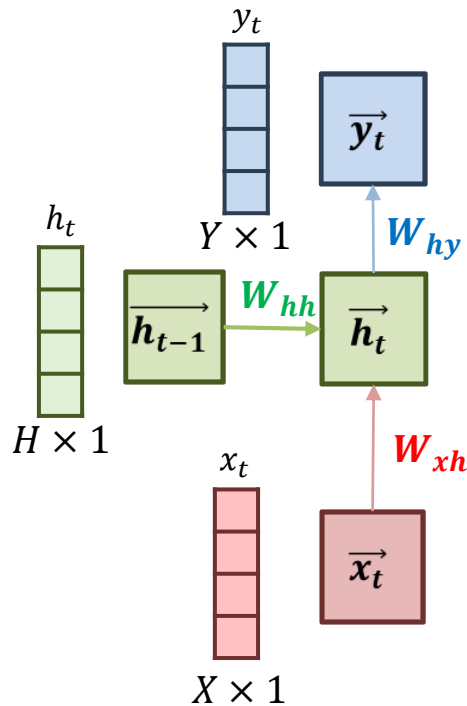
$\tanh$ 를 주로 사용하는 이유는?



RNN의 구조

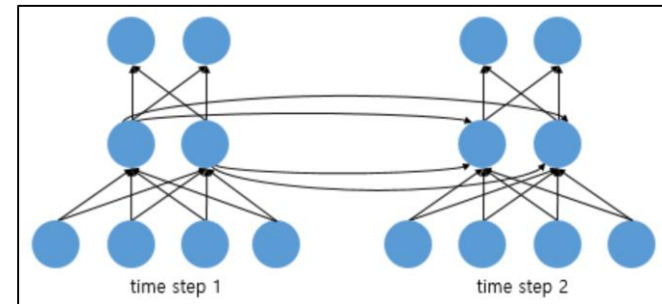


▪ h_t, y_t 계산의 벡터 표현



$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_n)$$

$$y_t = W_{hy}h_t + b_y$$



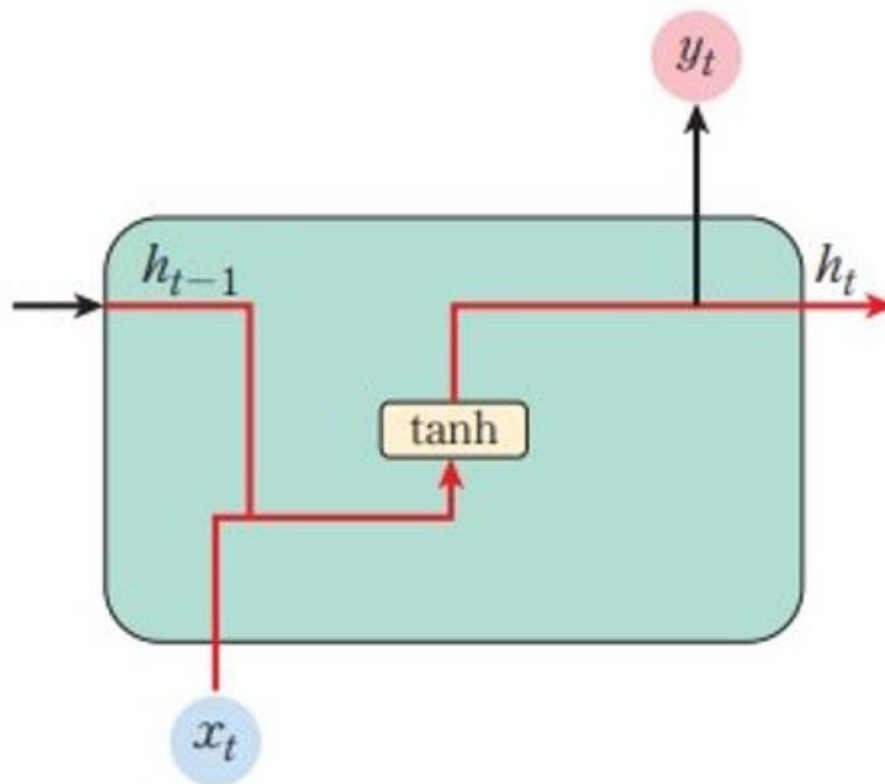
H : h_t 의 크기 (Hidden Unit 수)
 X : x_t 의 크기 (입력의 특징 수)
 Y : y_t 의 크기 (Hidden Unit 수와 동일)

$$h_t = \tanh \left(\begin{matrix} W_{hh} & h_{t-1} \\ H \times H & H \times 1 \end{matrix} \times + \begin{matrix} W_{xh} & x_t \\ H \times X & X \times 1 \end{matrix} + \begin{matrix} b_n \\ H \times 1 \end{matrix} \right)$$

[참고] RNN의 구조



- RNN 구조 가시화

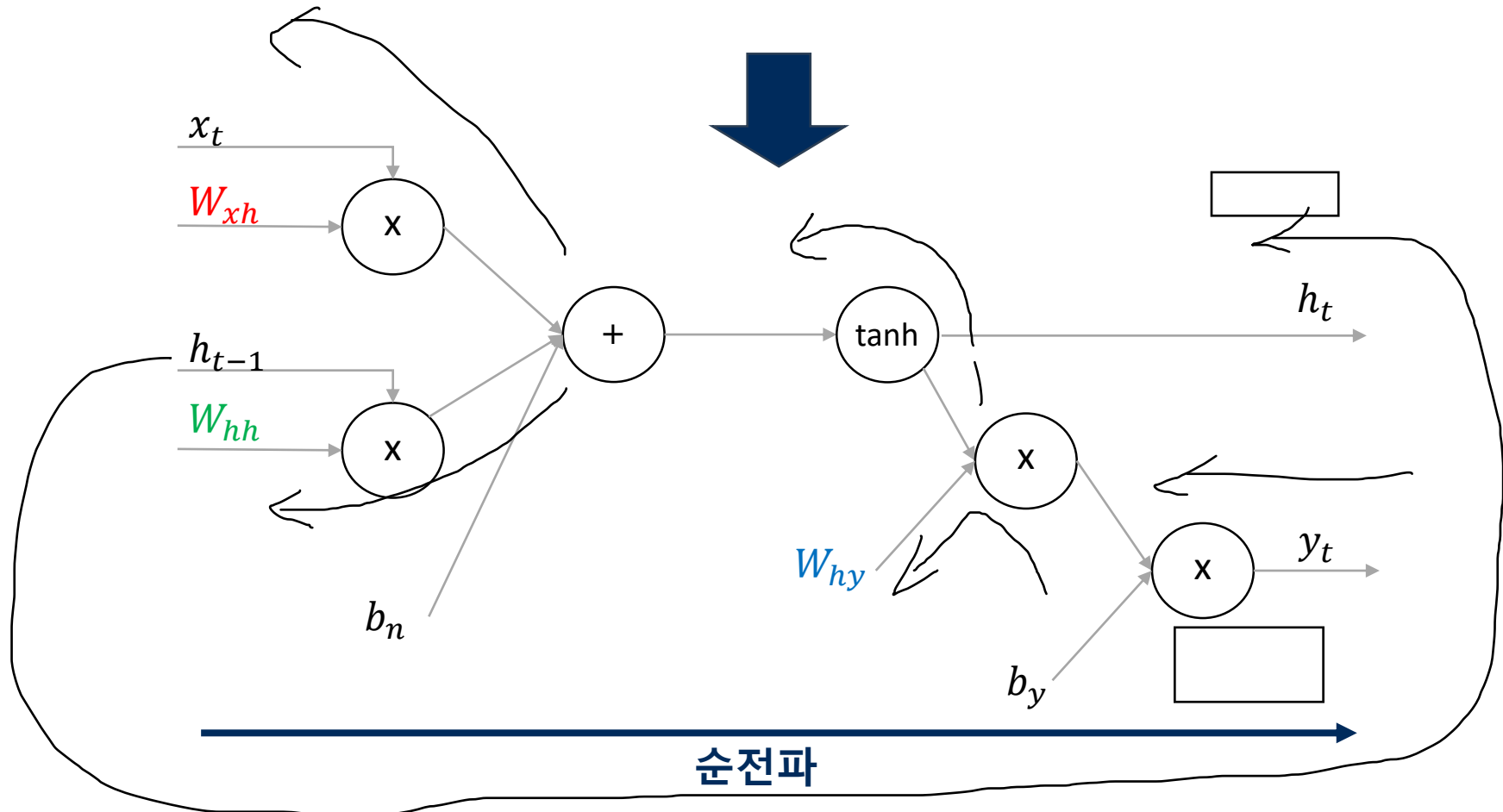


RNN 순전파

▪ RNN Forward-propagation

$$y_t = W_{hy}h_t + b_y$$

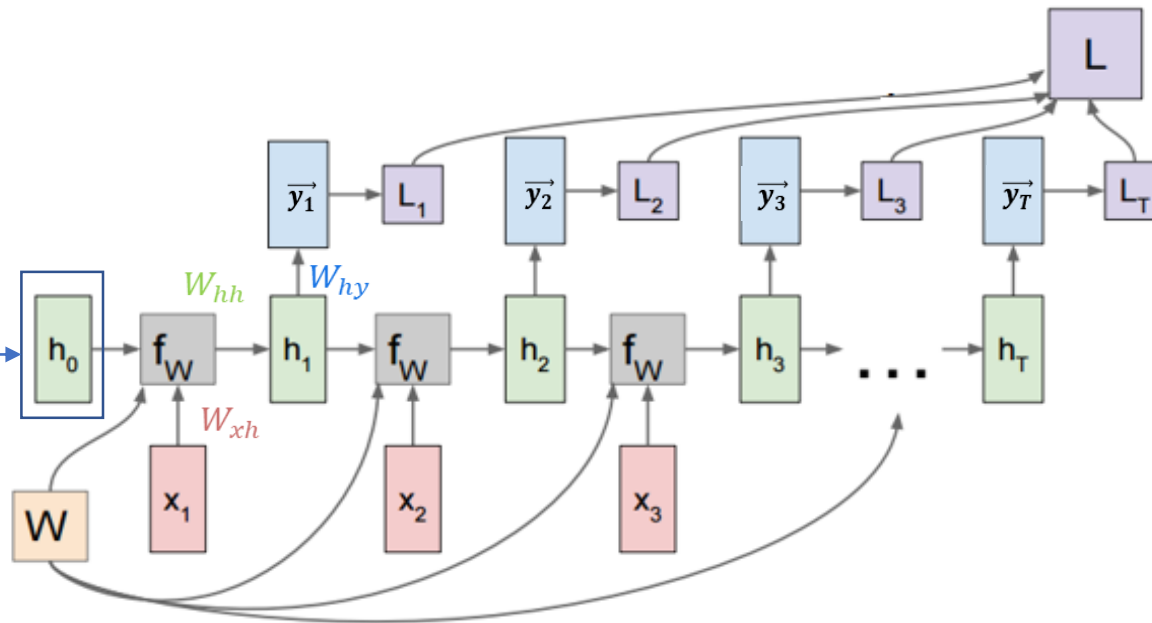
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_n)$$



RNN Training Process

■ RNN 학습 과정

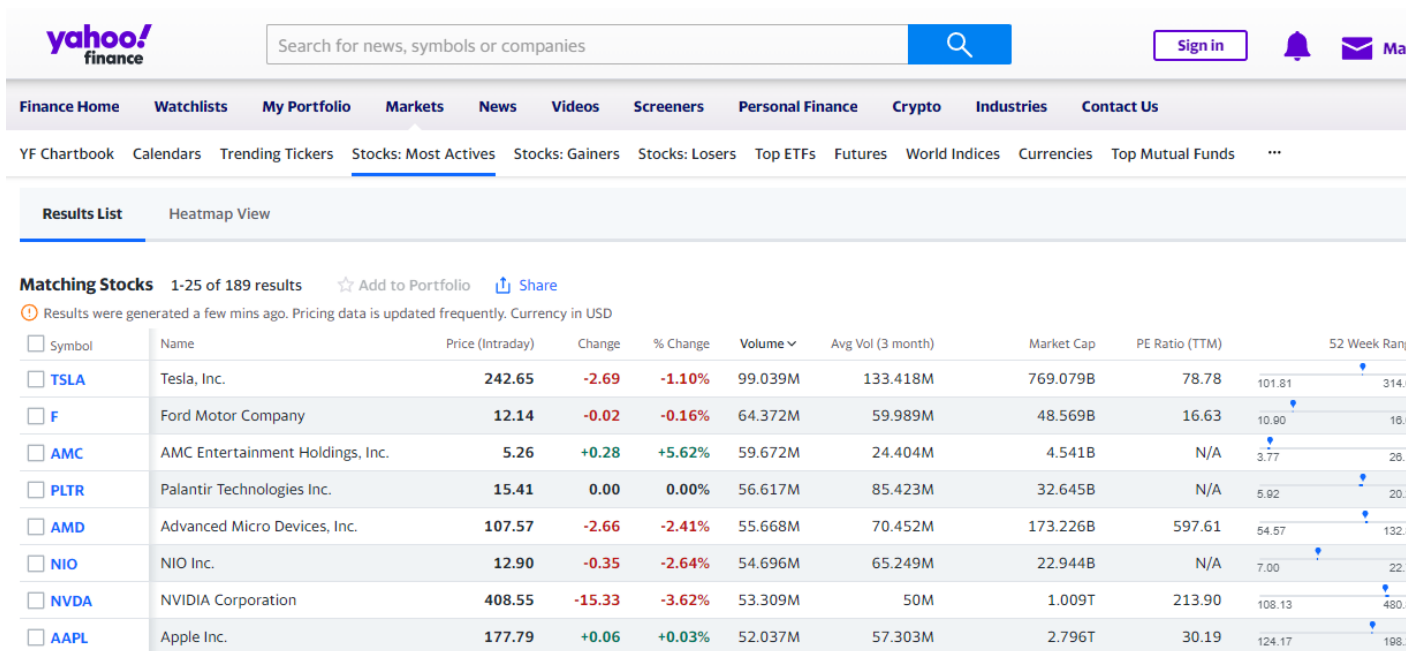
- 초기 hidden state(h_0)은 랜덤 값으로 고정하여 활용
- 모든 Weight(W)는 모든 시점에서 동일한 값
- Loss : 분류 문제 → Cross Entropy Error
회귀 문제 → MSE



Many-to-Many

RNN 실습 - 주가 예측

주가 정보 데이터 수집



The screenshot shows the Yahoo Finance website with the 'Matching Stocks' section. It displays a table of 1-25 of 189 results. The table includes columns for Symbol, Name, Price (Intraday), Change, % Change, Volume, Avg Vol (3 month), Market Cap, PE Ratio (TTM), and 52 Week Range. The stocks listed are Tesla, Inc., Ford Motor Company, AMC Entertainment Holdings, Inc., Palantir Technologies Inc., Advanced Micro Devices, Inc., NIO Inc., NVIDIA Corporation, and Apple Inc.

Symbol	Name	Price (Intraday)	Change	% Change	Volume	Avg Vol (3 month)	Market Cap	PE Ratio (TTM)	52 Week Range
TSLA	Tesla, Inc.	242.65	-2.69	-1.10%	99.039M	133.418M	769.079B	78.78	101.81 - 314.07
F	Ford Motor Company	12.14	-0.02	-0.16%	64.372M	59.989M	48.569B	16.63	10.90 - 16.08
AMC	AMC Entertainment Holdings, Inc.	5.26	+0.28	+5.62%	59.672M	24.404M	4.541B	N/A	3.77 - 26.16
PLTR	Palantir Technologies Inc.	15.41	0.00	0.00%	56.617M	85.423M	32.645B	N/A	5.92 - 20.24
AMD	Advanced Micro Devices, Inc.	107.57	-2.66	-2.41%	55.668M	70.452M	173.226B	597.61	54.57 - 132.83
NIO	NIO Inc.	12.90	-0.35	-2.64%	54.696M	65.249M	22.944B	N/A	7.00 - 22.74
NVDA	NVIDIA Corporation	408.55	-15.33	-3.62%	53.309M	50M	1.009T	213.90	108.13 - 480.88
AAPL	Apple Inc.	177.79	+0.06	+0.03%	52.037M	57.303M	2.796T	30.19	124.17 - 198.23

주식 정보 사이트 yahoo finance

- yahoo finance의 파이썬 API 설치



가상환경 진입



```
C:\Users\admin>pip install yfinance
```

자신의 가상환경에 설치!!

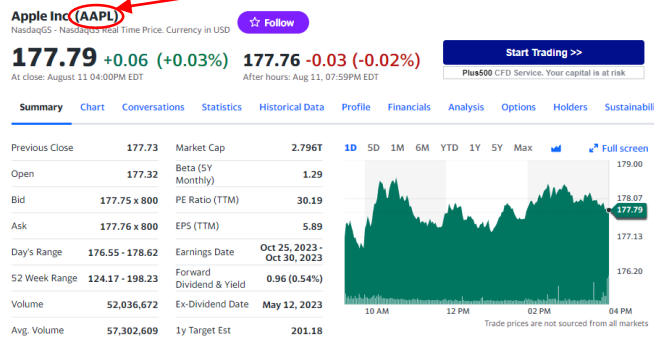
RNN 실습 - 주가 예측



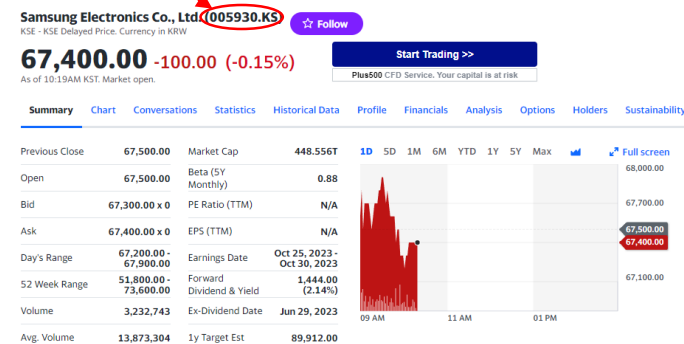
지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

- 주가 정보 데이터 수집
 - 주가정보 다운로드

종목코드 (Ticker)



애플 주식 정보



삼성전자 주식 정보

원하는 종목코드 입력

시작 날짜

```
import yfinance as yf
AppleFinance = yf.download('AAPL', start = '2021-01-01')
```

Date	Open	High	Low	Close	Adj Close	Volume
2021-01-04 00:00:00	133.52	133.61	126.76	129.41	127.332	143301900
2021-01-05 00:00:00	128.89	131.74	128.43	131.01	128.906	97664900
2021-01-06 00:00:00	127.72	131.05	126.38	126.6	124.567	155088000
2021-01-07 00:00:00	128.36	131.63	127.86	130.92	128.817	109578200

RNN 실습 – 주가 예측

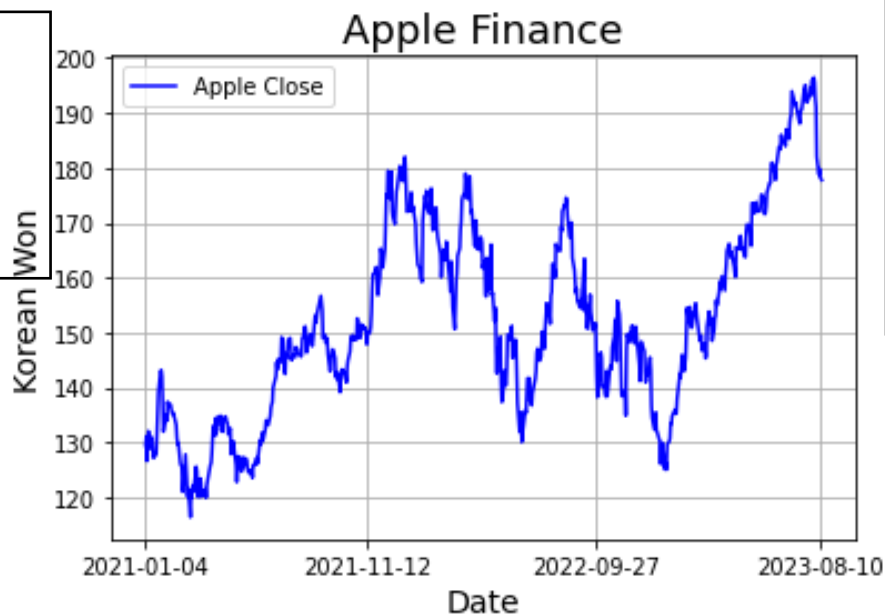


주가 정보 데이터 시각화

```
apl_close = np.array(AppleFinance['Close'])
date = AppleFinance.index.date
xtick = np.arange(0, date.shape[0], int(date.shape[0]/3))

plt.figure(1)
plt.plot(date, apl_close, label="Apple Close", color="blue")
plt.xticks(date[xtick])
plt.xlabel('Date', fontsize=14)
plt.ylabel('Dollar', fontsize=14)
plt.title('Apple Finance', fontsize=18)
plt.grid()
plt.legend()
```

10 10 11 () 12 23 (10 +1)



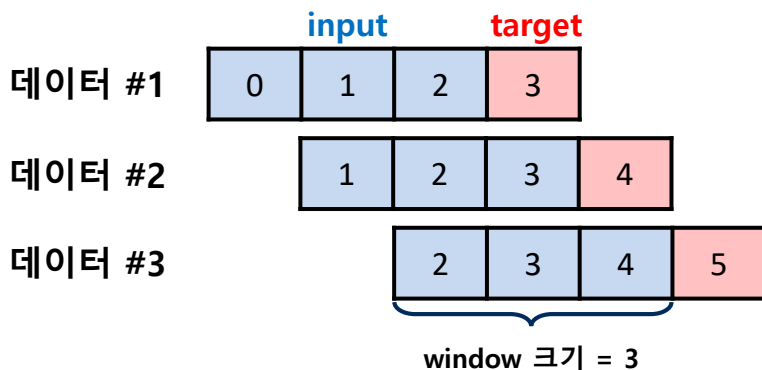
RNN 실습 – 주가 예측

■ 학습 데이터 전처리

- 훈련 데이터 제작

시계열 데이터

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---



window 크기 = 3인 경우



데이터 3개를 보고 다음 값 예측
train input = 3개
train target = 1개

```
window_size = 3
x_train = []
t_train = []

for i in range(len(apl_close) - window_size):
    x_train.append(apl_close[i:i+window_size])
    t_train.append(apl_close[i+window_size])
x_train = np.array(x_train)/np.max(x_train)
t_train = np.array(t_train)/np.max(t_train)
```

RNN 실습 - 주가 예측



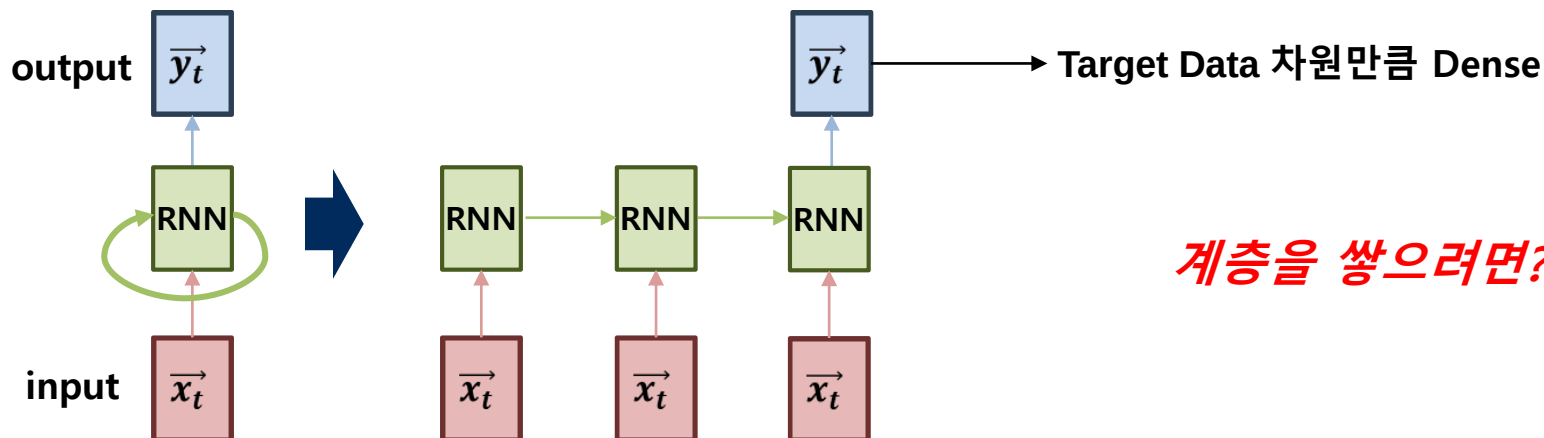
Train set 학습

- 학습 진행

```
import keras
model = keras.models.Sequential()
model.add(keras.layers.SimpleRNN(10, activation='tanh', input_shape=(3,1)))
model.add(keras.layers.Dense(1, activation='tanh'))
model.compile(optimizer='adam', loss='MSE')
history = model.fit(x_train, t_train, epochs=100)
```

Hidden State의 크기 결정 (Weight 개수에 관여)

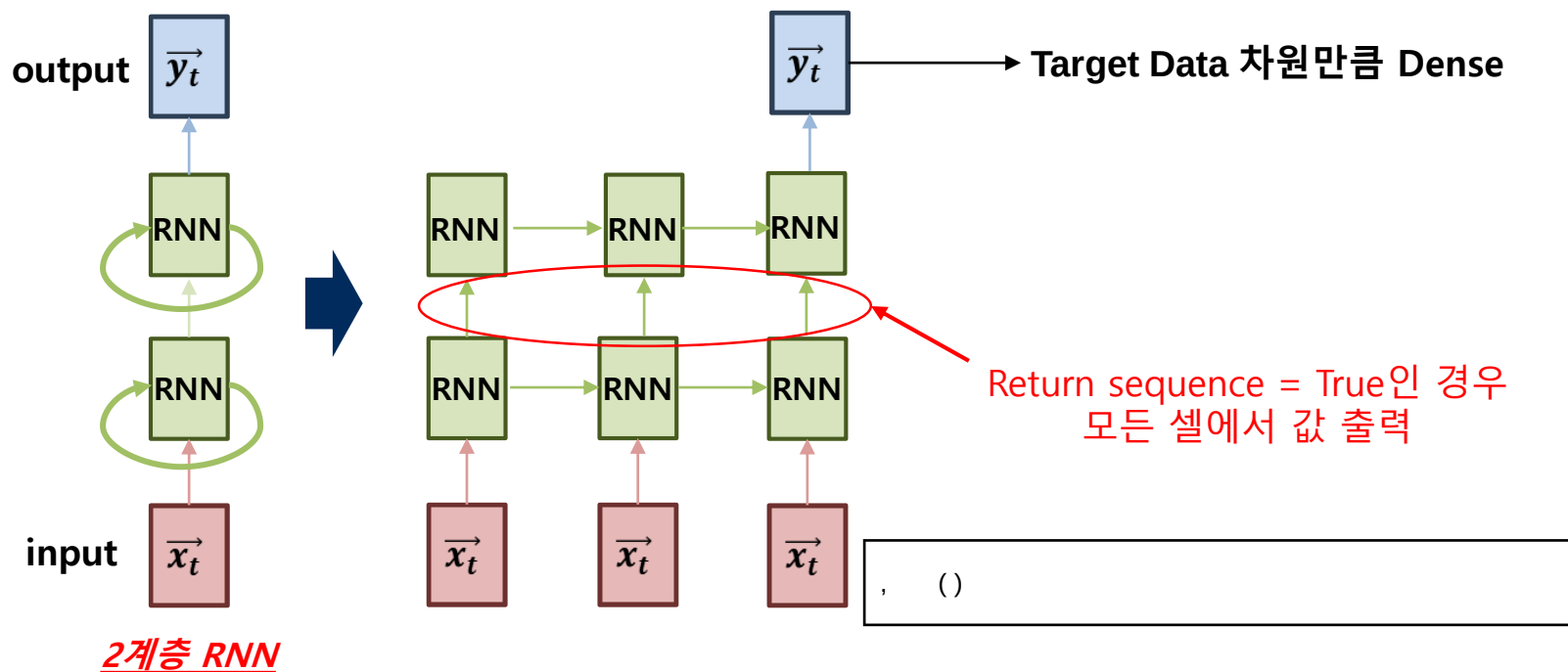
(time step, input dim)
ex) 50일치 증가, 거래량을 보는 경우 (50,2)



RNN 실습 – 주가 예측



- 다계층 RNN 구조
 - Return sequence



```
model = keras.models.Sequential()  
model.add(keras.layers.SimpleRNN(units=128, return_sequences=True, input_shape=(window_size, 1)))  
model.add(keras.layers.SimpleRNN(units=64))  
model.add(keras.layers.Dense(1, activation='tanh'))
```


▪ Test set 예측

- 2019년 ~ 2021년 데이터 수집

```
import yfinance as yf
test_AppleFinance = yf.download('AAPL', start = '2019-01-01', end = '2021-12-31')
test_apl_close = np.array(test_AppleFinance['Close'])
test_date = test_AppleFinance.index.date
```

- 데이터 전처리

```
window_size = 3
x_test = []
t_test = []

for i in range(len(test_apl_close) - window_size):
    x_test.append(test_apl_close[i:i+window_size])
    t_test.append(test_apl_close[i+window_size])

x_test = np.array(x_test)/np.max(x_test)
t_test = np.array(t_test)/np.max(t_test)
```

- Test 주가 예측

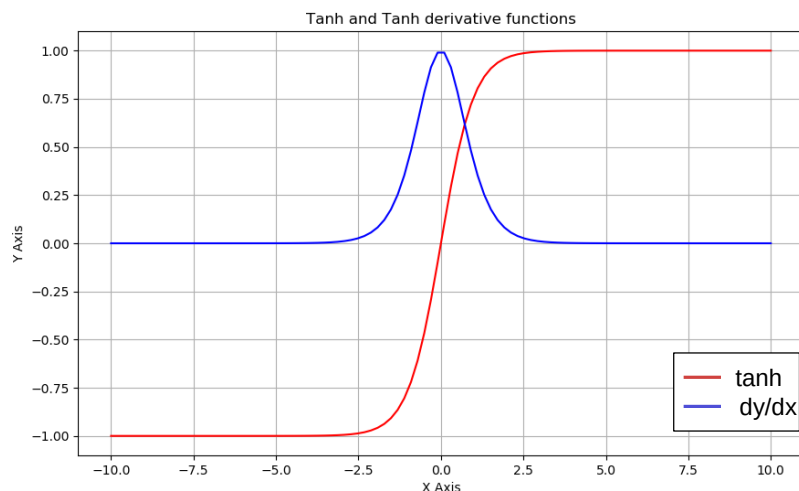
```
pred_test = model.predict(x_test)
```

▪ “장기 의존성” 문제 발생

- 시간이 지난 과거를 기억하기 위해서는 과거 방향으로 '의미 있는 기울기'를 전달함으로써 시간에 Dependent한 내용을 학습
- 데이터의 길이가 길어질수록(=정보와 정보 사이가 멀수록) 앞의 내용이 뒤로 전달되지 않는 문제 발생 → 기울기가 작아짐



▪ Gradient Vanishing

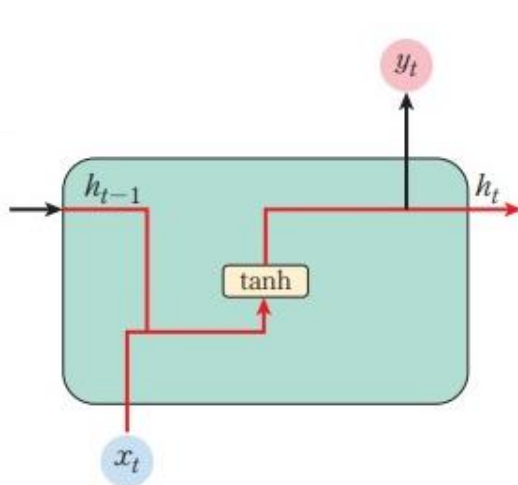


- x의 출력인 y가 다시 x로 들어가는 구조
- 즉, 기울기가 계속해서 작아질 수 있음
- 기하급수적으로 감소할 수 있는 곱셈 구조 때문에 “장기 의존성”을 유지하기 어려움

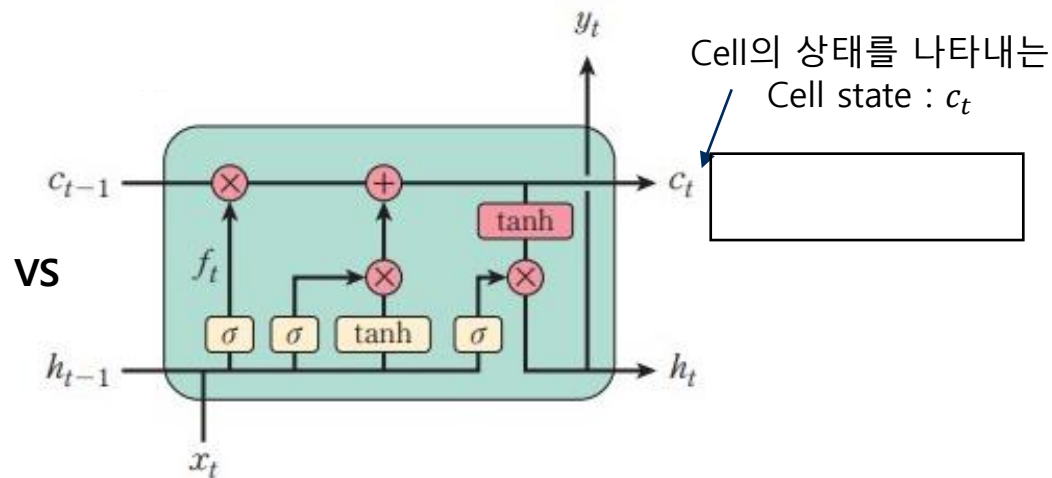
Long Short-Term Memory

▪ LSTM (장단기 기억 신경망)

- RNN의 장기 의존성 문제를 보완하기 위해 고안
- 장/단기 기억을 가능하게 설계한 신경망 구조
- LSTM Cell 구성요소
 - Forget Gate / Input Gate / Output Gate
- Cell은 임의의 시점에 대한 값을 기억
- 세 개의 Gate는 Cell로 들어오고 나가는 정보의 흐름을 조절



RNN Cell

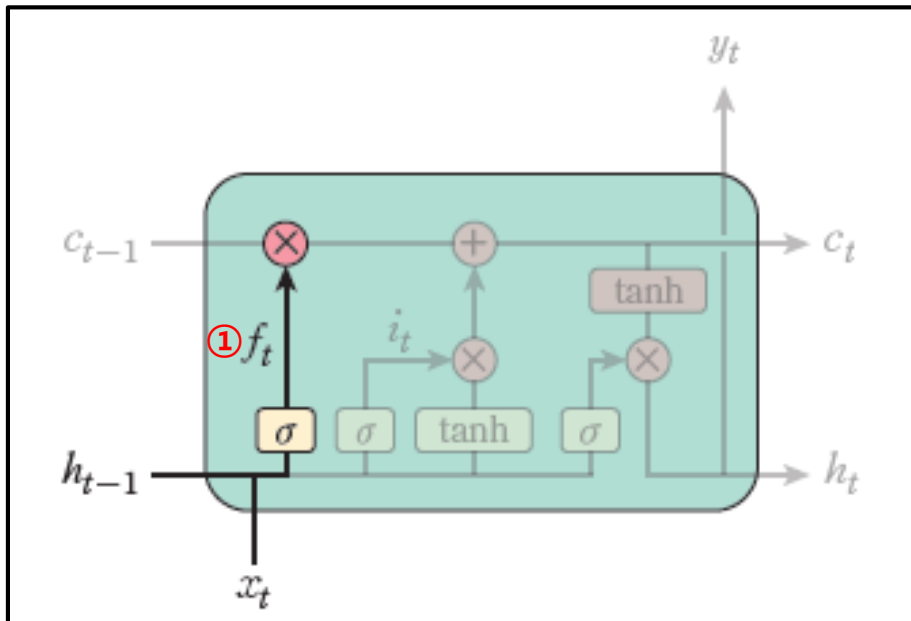


LSTM Cell

LSTM – Forget Gate

Forget Gate (망각 게이트)

- 과거 정보를 잊기 위한 게이트
- Forget Gate의 값이 0에 가까울 수록 많은 정보를 버리고, 1에 가까울 수록 많은 정보를 보존



$$\textcircled{1} f_t = \sigma(W_f \cdot [h_{t-1}, x_t])$$

f_t : Forget Gate의 값

h_{t-1} : 이전 시점의 Hidden State

σ : Sigmoid 함수

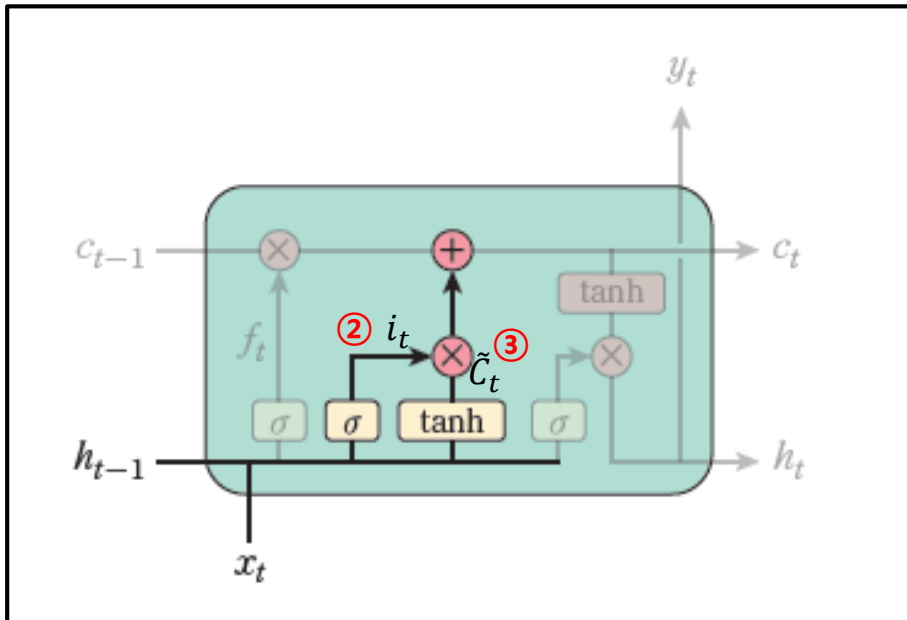
W_f : 각 게이트에 해당하는 Weight

(! 0)

LSTM – Input Gate

Input Gate (입력 게이트)

- Cell State에 새로운 정보를 저장하기 위한 게이트
- 새로운 정보들 중에 어떤 값을 Cell State에 저장할 것인지 결정



$$\textcircled{2} i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\textcircled{3} \tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

h_{t-1} : 이전 시점의 Hidden State

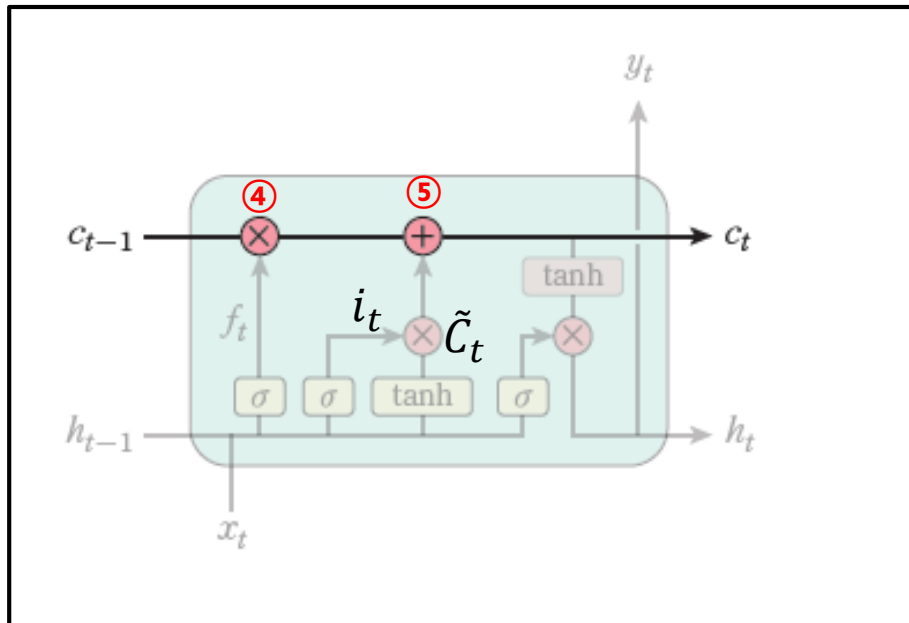
σ : Sigmoid 함수

W_i, W_c : Weight

LSTM – Cell State

Cell State (셀 상태)

- Cell State는 정보가 바뀌지 않고 그대로 흐르도록 하는 역할
- 일종의 컨베이어 벨트 역할



$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

C_{t-1} : 이전 시점의 Cell State
 C_t : 현재 시점의 Cell State

아다마르 곱 (Hadamard product) 이란? ($\odot$)

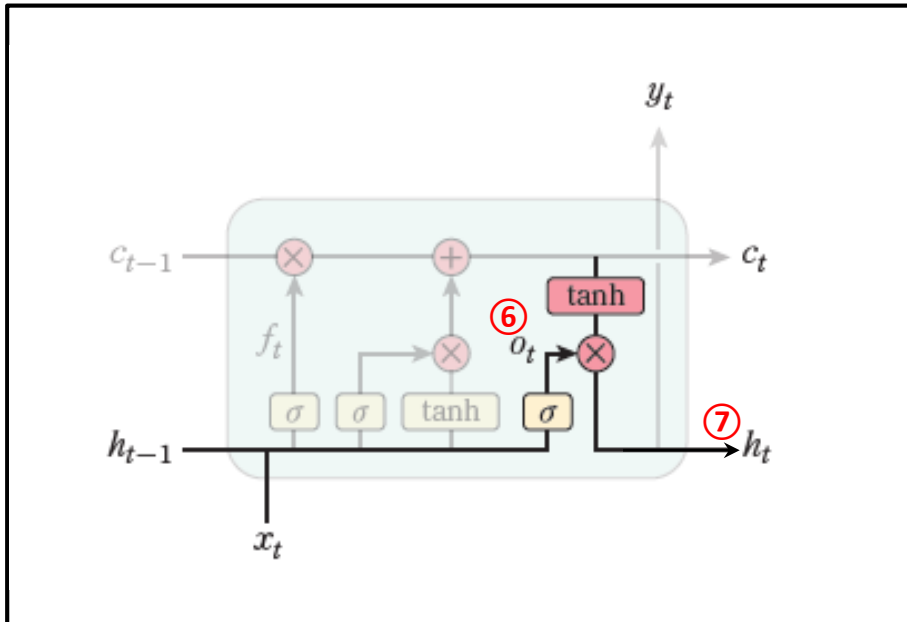
동일한 형상의 두 행렬이 있을 때, 각 원소별로 곱한 것

$$\begin{matrix} n \\ \begin{matrix} a_{11}, a_{12}, \dots, a_{1n} \\ a_{21}, a_{22}, \dots, a_{2n} \\ \vdots \\ a_{m1}, a_{m2}, \dots, a_{mn} \end{matrix} \end{matrix} \odot \begin{matrix} n \\ \begin{matrix} b_{11}, b_{12}, \dots, b_{1n} \\ b_{21}, b_{22}, \dots, b_{2n} \\ \vdots \\ b_{m1}, b_{m2}, \dots, b_{mn} \end{matrix} \end{matrix} = \begin{matrix} n \\ \begin{matrix} a_{11}b_{11}, a_{12}b_{12}, \dots, a_{1n}b_{1n} \\ a_{21}b_{21}, a_{22}b_{22}, \dots, a_{2n}b_{2n} \\ \vdots \\ a_{m1}b_{m1}, a_{m2}b_{m2}, \dots, a_{mn}b_{mn} \end{matrix} \end{matrix}$$

LSTM – Output Gate

Output Gate (출력 게이트)

- 어떤 정보를 다음 시점의 Hidden State를 제어하기 위한 게이트
- RNN과 달리 h_{t-1} 과 x_t 에 Sigmoid 적용 → 중요한 정도를 조절



$$\textcircled{6} \quad O_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$\textcircled{7} \quad h_t = O_t \odot \tanh C_t$$

h_{t-1} : 이전 시점의 Hidden State

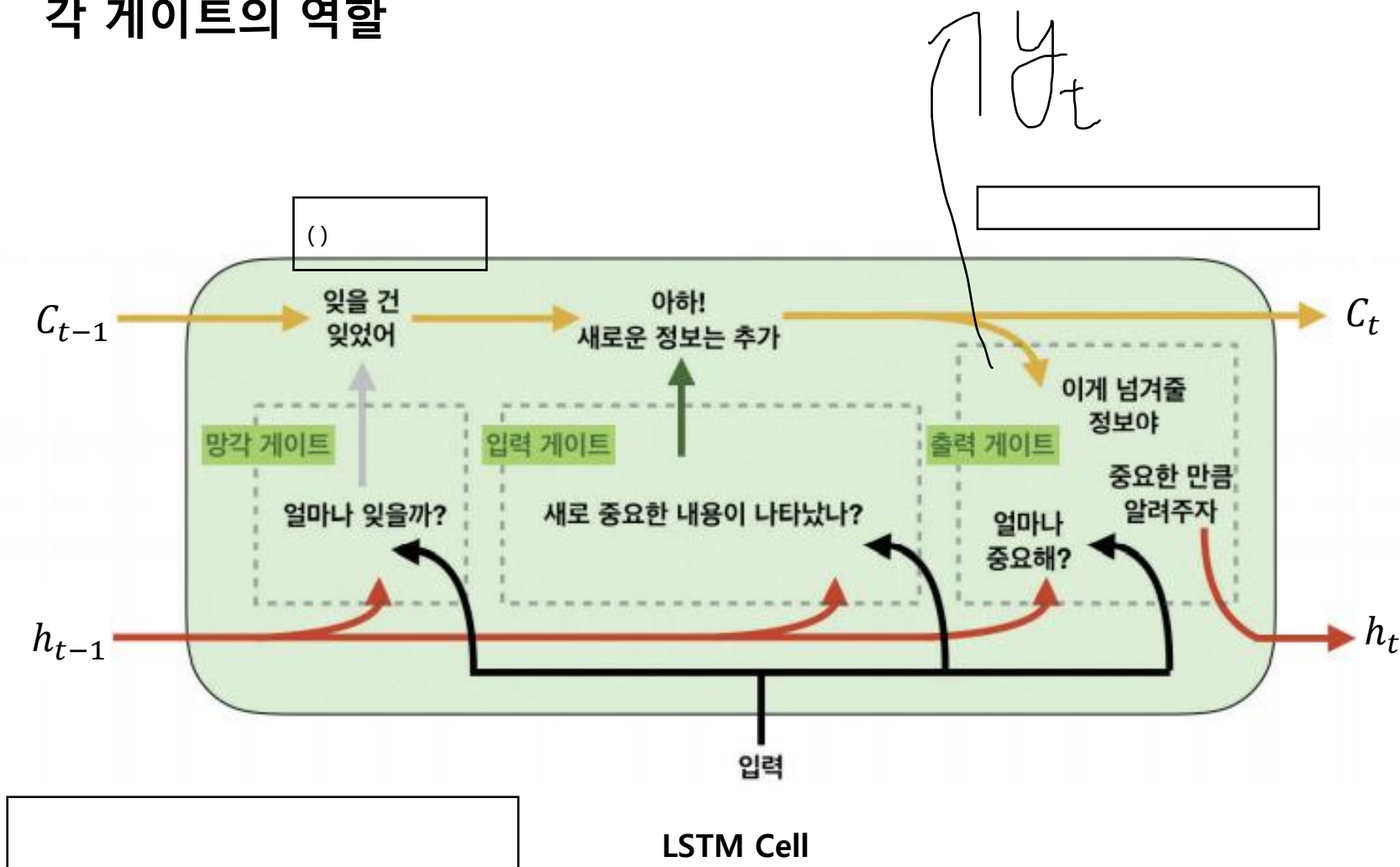
h_t : 현재 시점의 Hidden State

W_o : Weight

[참고] LSTM 구조 복습



■ 각 게이트의 역할



[참고] GRU (Gated Recurrent Unit)

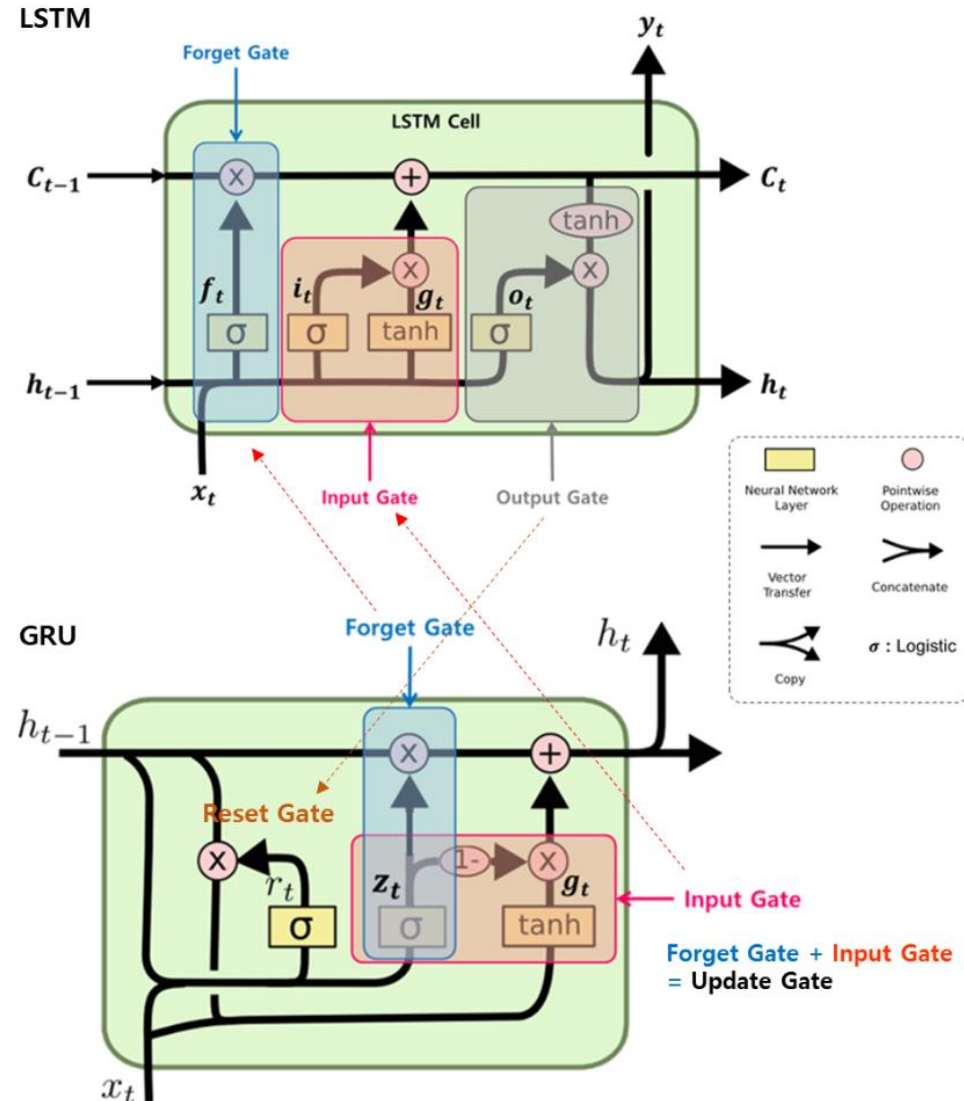


■ LSTM의 간소화 (연산속도 향상)

1. Cell State가 없음.
2. Hidden State가 Cell State 역할
3. Forget gate 및 Input gate가 통합
4. Output gate => Reset get

경험적으로,

- 데이터가 많을 때 => LSTM
- 데이터가 적을 때 => GRU



LSTM 실습



■ 모델 변경

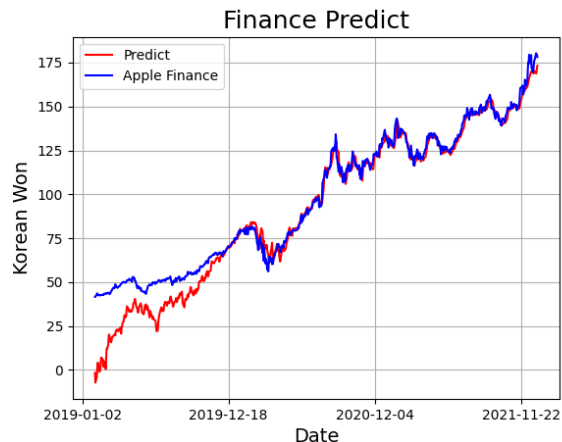
- RNN 실습과 동일하게 진행

```
import keras

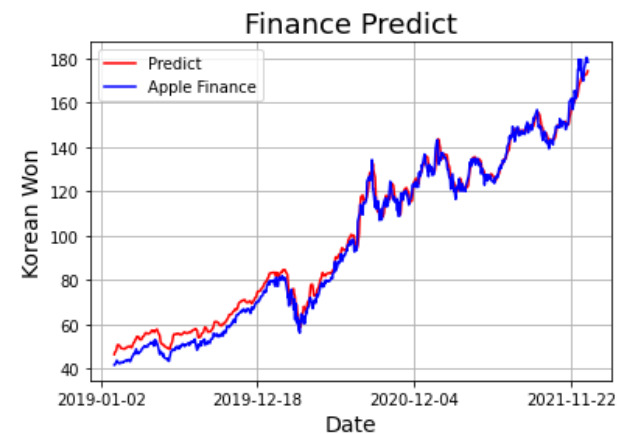
model = keras.models.Sequential()
model.add(keras.layers.LSTM(10, activation='tanh', input_shape=(window_size, 1)))
model.add(keras.layers.Dense(1, activation='tanh'))

model.compile(optimizer='adam', loss='MSE')
```

RNN 계층을 LSTM 계층으로 변경



RNN 실험 결과 예시

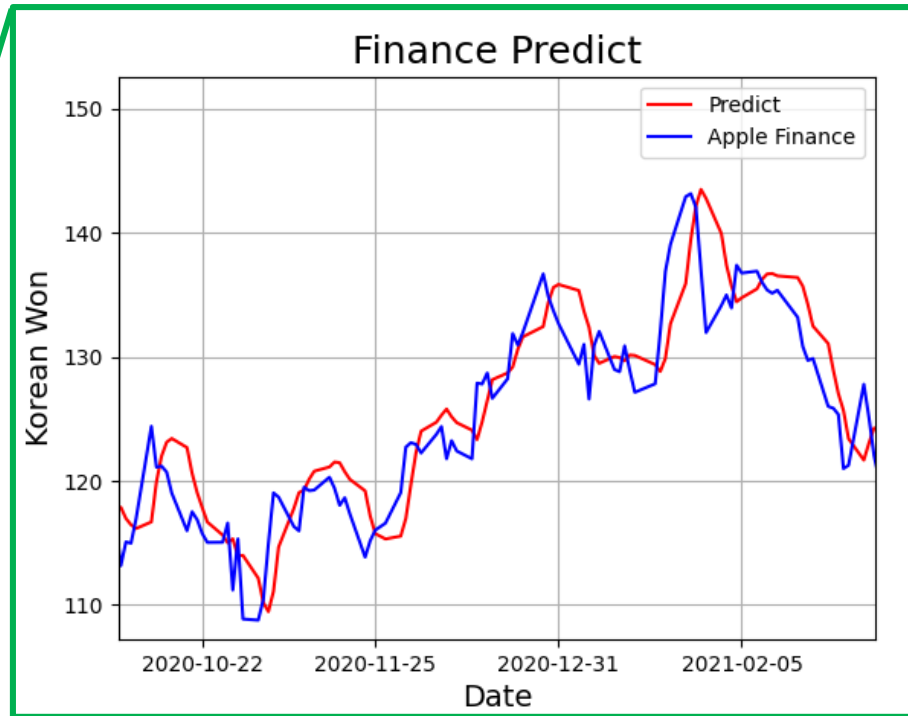
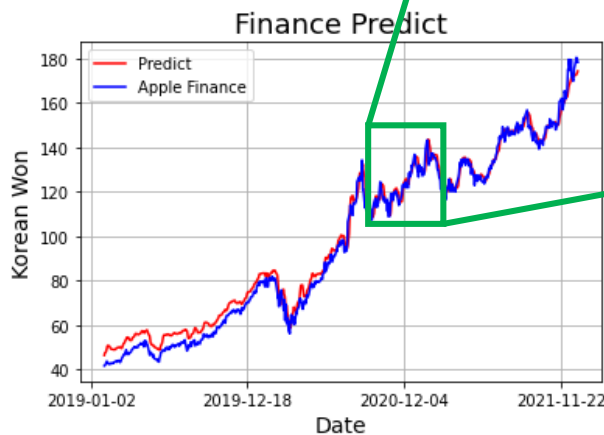


LSTM 실험 결과 예시

LSTM 실습



■ LSTM 실습



과연 정말 예측을 잘 했을까?

LSTM 실습



■ 주가예측 모델의 한계점

- 예측값이 실제값을 뒤따라가는 양상

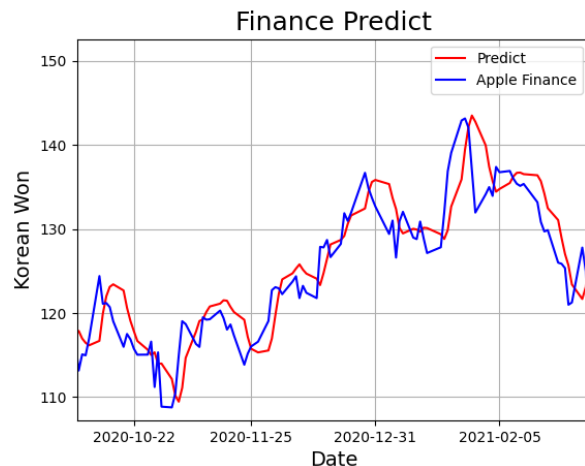
- 어떻게 하면 잘 예측할 수 있을까?

→ Close 값 하나만이 아닌 여러 종류의 값을 입력으로 학습
(High, Low, Volume 등등)

→ Convolution Layer를 활용하여 학습

→ 창의적인 아이디어로 직접 특징을 학습하여 LSTM 네트워크에 적용

개선
아이디어
예시



예측 결과의 양상

Date	Open	High	Low	Close	Adj Close	Volume
2021-01-04 00:00:00	133.52	133.61	126.76	129.41	127.332	143301900
2021-01-05 00:00:00	128.89	131.74	128.43	131.01	128.906	97664900
2021-01-06 00:00:00	127.72	131.05	126.38	126.6	124.567	155080000
2021-01-07 00:00:00	128.36	131.63	127.86	130.92	128.817	109570200
2021-01-08 00:00:00	132.43	132.63	130.23	132.05	129.929	105158200
2021-01-11 00:00:00	129.19	130.17	128.5	128.98	126.909	100384500
2021-01-12 00:00:00	128.5	129.69	126.86	128.8	126.731	91951100
2021-01-13 00:00:00	128.76	131.45	128.49	130.89	128.788	88636000
2021-01-14 00:00:00	130.8	131	128.76	128.91	126.84	90221800
2021-01-15 00:00:00	128.78	130.22	127	127.14	125.098	111598500
2021-01-19 00:00:00	127.78	128.71	126.94	127.83	125.777	90757300
2021-01-20 00:00:00	128.66	132.49	128.55	132.03	129.91	104319500
2021-01-21 00:00:00	133.8	139.67	133.59	136.87	134.672	120150900

데이터의 여러 특징

각 실습문제에 대해 code(+ 주석), 그래프, 분석 내용 등은 필수적으로 포함시킬 것

실습 #1) 앞선 실습에서 진행한 RNN 및 LSTM의 학습 파라미터 최적화

- yahoo finance를 활용하여 데이터 수집 (원하는 종목 코드를 골라 사용해도 무관)
- 자신만의 신경망 모델 디자인 (계층 추가, 활성화 함수 등등)
- 여러 **파라미터 조절**에 따른, 결과 비교 분석 (Learning rate, Cell 개수, Epoch, Window Size, Optimizer, Loss 등등)
- RNN 및 LSTM에 대한 Train set 및 Test set의 Loss값 및 예측 결과 비교 분석
- 조건 : Train set 및 Test set **기간의 합은 최소 4년**이 넘도록 할 것

실습 #2) 새로운 아이디어를 적용하여 주가 예측 모델의 한계점 개선

- 실습 #1에서 사용한 데이터와 파라미터는 변경하지 말 것
- 실습 #1의 결과와 자신만의 새로운 아이디어를 적용한 순환 신경망의 결과 비교 분석